

Orientation & Thesis: Closure of Compositional Hierarchies in Digital Symmetries

This chapter orients the reader to the book’s central thesis: that many digital systems admit a useful notion of *closure* under composition, and that the resulting compositional hierarchies are often organized by symmetry. The guiding question is not merely whether a given object can be built from simpler parts, but whether the class of admissible objects is stable under the operations used to build them, and whether the resulting structure can be analyzed up to the symmetries that preserve its behavior.

We will use the term *digital object* broadly to include Boolean functions on $\mathbb{B}^n$, circuits, finite automata, and codes. These objects appear in different guises across logic, hardware, software, and information theory, but they share a common feature: they are finite or finitely presented systems whose behavior can be described, transformed, and compared by explicit operations. In this setting, closure means that a chosen class of objects is preserved by the relevant constructions, while hierarchy refers to the stratification of objects by compositional depth, structural complexity, or refinement level.

A recurring theme of the book is that symmetry is not an afterthought but a structural constraint. We will study invariance and equivariance under permutations Σ_n , under input and output negations, and under the combined action that gives rise to NPN classes. These symmetry notions organize Boolean functions and circuits into equivalence classes that are often more informative than raw syntactic descriptions. In later chapters, the same perspective will be extended to automata, dataflow systems, and other compositional formalisms.

Composition is the second organizing principle. The book will repeatedly use three basic operations: sequential composition $\circ$, parallel composition $\otimes$, and feedback or trace Tr . The intended laws are familiar but essential: associativity for sequential and parallel composition, symmetry for interchange of components, and functoriality or naturality conditions that ensure compatibility between structure-preserving maps and the operations themselves. These laws provide the algebraic backbone for the constructions developed throughout the book.

To compare digital objects at different levels of description, we will also use error and discrepancy measures. At the level of Boolean functions, $\varepsilon(r)$ may denote truth-table Hamming error at resolution r ; at the level of circuits, it may denote a structural edit distance for wiring at resolution r . The precise form of ε will vary with context, but the underlying idea is consistent: a hierarchy is meaningful only if we can quantify how far an object lies from another object, or from a target specification, at the chosen level of abstraction.

The scope of this chapter is intentionally foundational. It does not attempt to prove the deepest classification theorems or to develop every formalism in full generality. Instead, it establishes the vocabulary and the thesis that will govern the rest of the book: digital systems are best understood as compositional objects equipped with symmetries, and the central technical problem is to determine when the resulting classes are closed under the operations that generate them. Once this problem is posed clearly, the later chapters can specialize it to operads, PROPs, clones, modular design, feedback, and applications in computation and engineering.

In short, the book asks three questions throughout. What are the relevant digital objects? Which symmetries identify behaviorally equivalent forms? And under which composition laws do these objects form closed hierarchies? The chapters that follow develop these questions into precise definitions, constructions, and examples.

For orientation, the next chapter develops the categorical and algebraic foundations of composition itself, including operads, PROPs, and symmetric monoidal structure. That material will

supply the formal language needed to state closure properties precisely and to compare different notions of hierarchy across the book's later case studies.

For the purposes of this book, *closure* will mean more than mere closure under a single operation. A class of digital objects is closed when the operations that generate, combine, or transform its members do not force us to leave the class in order to describe the result. Likewise, *hierarchy* will mean more than a simple ordering by size: it will refer to a layered organization in which objects at one level are assembled from objects at lower levels, and where the passage between levels is controlled by explicit composition laws and symmetry relations. These two ideas—closure and hierarchy—will recur throughout the book as the main criteria for deciding whether a formalism is robust enough to support analysis, synthesis, and comparison.

The chapters ahead will therefore proceed in a deliberately cumulative way. We begin with the abstract language of composition, then specialize to Boolean functions and their clone-theoretic closures, and then move outward to circuits, automata, modular design, and symmetry-aware applications. The unifying claim is that the same structural questions reappear in each setting: what is preserved, what is identified, and what is generated when digital systems are composed?